OAVS • TGT COMPUTER SCIENCE TEACHER EXAM

MCQ Question Bank

Pointers — Complete Topic Coverage

Concept of Pointers to Command Line Arguments

200 Unique, Exam-Oriented MCQs

13 Sub-Topics • Answer Key & Explanation for Every Question

Definition • Syntax • Code-Snippet • Output-Based • Assertion-Reason • Match-the-Following • Scenario-Based

Prepared strictly as per the OAVS TGT Computer Science Syllabus — Topic: Pointers

Table of Contents

Chapter-wise organization · 200 total MCQs · Topic: Pointers

1. Concept of Pointers ..... (16 MCQs)
2. Pointer as an Address Variable ..... (14 MCQs)
3. Pointer Data Type and Syntax ..... (16 MCQs)
4. Built-in Address Operator (&) ..... (14 MCQs)
5. Pointers to Variables of Different Data Types ..... (16 MCQs)
6. Indirection Operator (*) ..... (16 MCQs)
7. Array Name as a Pointer Variable ..... (14 MCQs)
8. Pointers and Arrays ..... (16 MCQs)
9. Pointer Arithmetic ..... (18 MCQs)
10. Unary Operators (++, --) with Pointers ..... (14 MCQs)
11. One-Dimensional Arrays and Pointers ..... (16 MCQs)
12. Array of Pointers ..... (14 MCQs)
13. Command Line Arguments for main() ..... (16 MCQs)

Chapter 1: Concept of Pointers

16 MCQs

Q1 | **DEFINITION** | ***Definition***

A pointer in C is a variable that:

A Stores the size of a data type

B Stores the memory address of another variable

C Stores a constant literal value

D Stores multiple values of the same type

Ans: B

Explanation: A pointer variable holds the memory address where a value is stored, not the value itself.

Q2 | **CONCEPT** | ***Concept***

The value stored in a pointer variable is called its:

A Content

B Index

C Identifier

D Address

Ans: D

Explanation: The value held by a pointer variable is the address of the variable it points to.

Q3 | **CONCEPT** | ***Concept***

Which of the following best describes the "content" pointed to by a pointer variable?

A The data type of the pointer

B The size of the pointer variable

C The value stored at the memory address it holds

D The name of the variable it points to

Ans: C

Explanation: Dereferencing a pointer gives the value stored at the address it holds — this is its 'content'.

Q4 | **CONCEPT** | ***Characteristics***

Which of these is a characteristic of pointer variables in C?

A Pointer size always varies with the type it points to

B Pointers must always point to arrays only

C Pointer variables cannot be reused

D On a given machine, all data-pointer types generally occupy the same fixed size in memory

Ans: D

Explanation: Since a pointer stores only an address, its own size is typically fixed for a given system/architecture.

Q5 | **CONCEPT** | ***Purpose***

What is the main purpose of using pointers in C programs?

A To avoid using arithmetic operators

B To replace all data types in a program

C To allow indirect access and manipulation of memory/variables

D To increase the readability of source code only

Ans: C

Explanation: Pointers provide a mechanism for indirect access to data through memory addresses.

Q6 | **STATEMENT** | ***Advantage***

Which of the following is an advantage of using pointers?

A They eliminate the need for variable declarations

B They make programs immune to runtime errors

C They allow efficient handling of arrays and strings through direct memory access

D They automatically increase execution speed of every program

Ans: C

Explanation: Pointers enable efficient traversal and manipulation of arrays/strings via address-based access.

Q7 | APPLICATION | *Application*

Pointers are commonly used in C for all of the following EXCEPT:

A Defining new data types like enum

B Efficient traversal of array elements

C Passing arguments indirectly through addresses

D Efficient handling of arrays

Ans: A

Explanation: Defining an enum does not involve pointers; the other options are classic pointer-based applications.

Q8 | SYNTAX | *Characteristics*

A pointer variable in C must be declared with:

A An asterisk (*) before its name, indicating the type it points to

B No type information at all

C An ampersand (&) before its name

D The keyword 'ptr' as a mandatory prefix

Ans: A

Explanation: The syntax type *name; declares a pointer of the given base type.

Q9 | STATEMENT-BASED | *Statement*

Which statement about pointers is TRUE?

A A pointer always holds an integer value

B A pointer variable can point to a variable of any compatible data type

C A pointer variable can only be declared inside main()

D A pointer variable cannot be reassigned once initialized

Ans: B

Explanation: A pointer can be reassigned any time to point to a different, type-compatible variable.

Q10 | TRICKY | *Characteristics*

Which of the following is NOT stored by a pointer variable?

A The value of the variable it points to

B A NULL value

C The address of another pointer

D The memory address of a variable

Ans: A

Explanation: A pointer stores an address, not the actual value of the pointed-to variable.

Q11 | SCENARIO | *Application*

In embedded and system-level programming, pointers are widely used mainly because they:

A Automatically manage program memory

B Simplify mathematical calculations only

C Provide direct and efficient access to memory locations

D Are the only way to declare variables

Ans: C

Explanation: System-level programs need direct, efficient memory access, which pointers provide.

Q12 | DEFINITION | *Characteristics*

A pointer that does not point to any valid memory location is called a:

A Void pointer

B Static pointer

C Wild pointer

D NULL pointer

Ans: D

Explanation: A NULL pointer is explicitly assigned to point to nothing (commonly represented as address 0).

Q13 | TRICKY | *Characteristics*

A pointer variable that has been declared but not assigned any valid address is known as a:

A Static pointer

B Void pointer

C Wild pointer

D NULL pointer

Ans: C

Explanation: An uninitialized pointer holding a garbage/indeterminate address is called a wild pointer.

Q14 | CONCEPT | *Concept*

Which pair of operators is fundamental to pointer operations in C?

A % and / only

B & (address-of) and * (dereference)

C && and || only

D + and - only

Ans: B

Explanation: The address-of (&) and indirection (*) operators form the core of pointer operations.

Q15 | ASSERTION-REASON | *Concept*

Assertion (A): Pointers increase the efficiency of a C program.
Reason (R): Pointers allow direct access to memory addresses, reducing the need for copying large data.

A A is true, R is false

B Both A and R are false

C A is false, R is true

D Both A and R are true, and R correctly explains A

Ans: D

Explanation: Direct memory access via pointers avoids costly copying of data, which correctly explains the efficiency gain.

Q16 | COMPARISON | *Comparison*

How does a pointer variable differ from a normal (non-pointer) variable?

A A normal variable can also directly store addresses without being declared as a pointer

B There is no difference between the two

C A pointer always requires more memory than any normal variable

D A pointer stores an address, while a normal variable stores a data value

Ans: D

Explanation: The fundamental distinction is that pointers hold addresses while ordinary variables hold data values.

Chapter 2: Pointer as an Address Variable

14 MCQs

Q17 | CONCEPT | *Memory Address*

Every variable in a C program is stored in memory at a:

A Unique address

B Fixed value

C Random index

D Sequential number starting from 0

Ans: A

Explanation: Each variable occupies a distinct memory location identified by a unique address.

Q18 | DEFINITION | *Memory Address*

The address of a variable is best described as:

A The size of the variable in bytes

B The data type of the variable

C The location in memory where the variable's value is stored

D The value assigned to the variable

Ans: C

Explanation: An address identifies the memory location holding the variable's value.

Q19 | STATEMENT | *Address Representation*

Which statement about memory addresses is correct?

A Memory addresses are typically displayed in hexadecimal form when printed

B Every variable in a program shares the same address

C Memory addresses are always shown in decimal only

D Memory addresses can be freely modified by the programmer

Ans: A

Explanation: printf with the %p format specifier conventionally displays addresses in hexadecimal.

Q20 | CONCEPT | *Address Storage*

A pointer variable can be used to:

A Store the address of another variable

B Replace loop control variables

C Store the name of a variable as text

D Store only integer constants

Ans: A

Explanation: This is the basic role of a pointer — holding the address of another variable.

Q21 | STATEMENT | *Pointer Initialization*

Which of these statements about pointer initialization is TRUE?

A A pointer can be safely initialized with any random integer

B A pointer must always be initialized to the value 100

C Pointer initialization is never necessary before use

D A pointer should be initialized with the address of a variable of a compatible type

Ans: D

Explanation: Using an uninitialized or incompatible pointer leads to undefined behaviour; proper, type-matched initialization is required.

Q22 | CODE SNIPPET | *Address Storage*

In the statement `int a = 10; int *p = &a;`, what does `p` hold?

A The address of variable a

B The value 10

C The data type int

D The address of pointer p itself

Ans: A

Explanation: p is assigned the address-of expression &a, so it holds a's address.

Q23 | FILL-IN-THE-BLANK | *Address Storage*

To assign the address of variable `x` to pointer `p`, the correct statement is: `p = ____ x;`

A &

B *

C %

D ++

Ans: A

Explanation: The & operator retrieves the address of a variable.

Q24 | CODE SNIPPET | *Address Storage*

Which of the following correctly stores the address of variable `num` into pointer `ptr`?

A ptr = num;

B ptr = *num;

C ptr = #

D *ptr = num;

Ans: C

Explanation: The & operator must be used to obtain num's address before assigning it to the pointer.

Q25 | TRICKY | *Pointer Initialization*

If a pointer variable is not assigned any address, its value is:

A Always equal to the address of main()

B Always zero automatically

C Always equal to -1

D Undefined/garbage until it is initialized

Ans: D

Explanation: An uninitialized local pointer contains an indeterminate (garbage) value.

Q26 | SYNTAX | *Address Representation*

Address representation of a variable is generally displayed using the format specifier:

A %s

B %f

C %d

D %p

Ans: D

Explanation: %p is the standard printf specifier used to print pointer/address values.

Q27 | ASSERTION-REASON | *Pointer Assignment*

Assertion (A): In C, a pointer variable can be assigned the address of a variable of any data type without restriction.
Reason (R): The compiler enforces type compatibility between a pointer and the address it stores, for type safety.

A A is false, R is true

B Both A and R are true, and R explains A

C Both A and R are true, but R does not explain A

D Both A and R are false

Ans: A

Explanation: Assigning an address of a mismatched type generally requires a cast; the compiler does enforce type compatibility, so A is false and R is true.

Q28 | TRICKY | *Pointer Assignment*

Which of the following pointer assignments is INVALID without an explicit typecast (assuming standard declarations)?

A int *p; p = 0;

B int *p; int *q; p = q;

C int *p; float f; p = &f;

D int *p; int a; p = &a;

Ans: C

Explanation: Assigning the address of a float variable to an int pointer is a type mismatch and needs a cast.

Q29 | CONCEPT | *Pointer Assignment*

Two pointer variables of the same type can:

A Be assigned to each other, copying the address they hold

B Never be assigned to each other

C Only be compared using arithmetic operators

D Never be compared to each other

Ans: A

Explanation: Same-type pointers can be directly assigned and compared.

Q30 | CONCEPT | *Address Storage*

The address stored in a pointer variable changes when:

A The value at the pointed variable changes

B The program is recompiled without any code changes

C The pointer variable's data type is printed

D The pointer is reassigned to point to a different variable

Ans: D

Explanation: Reassignment (e.g., p = &b;) is what changes the address a pointer stores.

Chapter 3: Pointer Data Type and Syntax

16 MCQs

Q31 | SYNTAX | *Syntax*

The general syntax for declaring a pointer variable in C is:

A datatype *pointer_name;

B *datatype pointer_name;

C datatype pointer_name*;

D pointer datatype_name;

Ans: A

Explanation: A pointer declaration places * before the identifier, after the base data type.

Q32 | CONCEPT | *Pointer Declaration*

In the declaration `int *p;`, the data type of `p` is:

A Pointer to char

B float

C Pointer to int

D int

Ans: C

Explanation: p's declared type is 'pointer to int', distinct from a plain int variable.

Q33 | SYNTAX | *Pointer Declaration*

Which of the following is a VALID pointer declaration?

A float *fp;

B float fp&;

C float fp*;

D *float fp;

Ans: A

Explanation: The asterisk must precede the identifier name in a valid declaration.

Q34 | **TRICKY** | ***Pointer Declaration***

Which of the following is an INVALID pointer declaration?

A int p*;

B int * p;

C int* p;

D int *p;

Ans: A

Explanation: The asterisk must appear before the identifier; 'int p*;' is syntactically incorrect (spacing around * is otherwise flexible).

Q35 | **CODE SNIPPET** | ***Pointer Declaration***

In `char *cp, c;`, which variable is a pointer?

A Both cp and c

B Only c

C Neither cp nor c

D Only cp

Ans: D

Explanation: In a comma-separated declaration list, * binds only to the identifier immediately following it.

Q36 | **CONCEPT** | ***Pointer Data Types***

What does the base data type in a pointer declaration (e.g., int in `int *p`) indicate?

A The number of variables the pointer can store

B The type of variable/data the pointer is intended to point to

C The value stored at the address, without needing dereferencing

D The size of the pointer variable itself

Ans: B

Explanation: The base type tells the compiler how to interpret the data found at the address the pointer holds.

Q37 | CODE SNIPPET | *Pointer Initialization*

Which statement correctly declares and initializes an integer pointer?

A int p = &a;

B int a = 5, *p = &a;

C int *p = a;

D int *p = 5;

Ans: B

Explanation: p must be assigned an address (&a), not a plain value.

Q38 | COMPARISON | *Pointer Data Types*

Pointer variables declared for different data types (int*, float*, char*) mainly differ in:

A Whether they can be printed using printf

B Their ability to store addresses at all

C The number of bytes the pointer variable itself occupies, which usually varies drastically by type

D How they interpret data at the pointed address and how far they move during pointer arithmetic

Ans: D

Explanation: The pointer's own storage size is typically fixed per system, but its base type governs data interpretation and arithmetic step size.

Q39 | DEFINITION | *Pointer Data Types*

Which keyword, when used with a pointer declaration, indicates it can point to a variable of any data type?

A void

B int

C extern

D static

Ans: A

Explanation: void* is C's generic pointer type, usable (with casting) for any data type.

Q40 | CONCEPT | *Pointer Data Types*

The declaration `double *dp;` means:

A dp stores the value of a double variable directly

B dp is a pointer that stores two addresses

C dp is itself a double variable

D dp is a pointer that will store the address of a double variable

Ans: D

Explanation: dp is declared as a pointer to double, meant to hold the address of a double.

Q41 | SYNTAX | *Pointer Declaration*

Which of the following correctly declares a pointer to a character variable?

A char cptr*;

B ptr char cptr;

C character *cptr;

D char *cptr;

Ans: D

Explanation: This is the standard syntax for declaring a character pointer.

Q42 | MATCH-THE-FOLLOWING | *Pointer Declaration*

Match the pointer declarations (Column A) with what they point to (Column B):
A) int *p B) float *q C) char *r
1. Character 2. Integer 3. Float

A A-2, B-1, C-3

B A-2, B-3, C-1

C A-1, B-2, C-3

D A-3, B-1, C-2

Ans: B

Explanation: int *p points to an int, float *q to a float, and char *r to a character.

Q43 | TRICKY | *Pointer Declaration*

Declaring `int *p, q;` means:

A p is a pointer to int, and q is a normal int variable

B Both p and q are pointers to int

C p is a normal int, and q is a pointer to int

D Both p and q are normal int variables

Ans: A

Explanation: The * applies only to the identifier it directly precedes, not to the whole declaration list.

Q44 | CONCEPT | *Pointer Initialization*

A pointer declared as `int *p;` without initialization contains:

A Automatically the address of a variable named p

B The value of the last used variable in the program

C An undefined/garbage address

D The value 0 always

Ans: C

Explanation: Local pointers in standard C are not automatically initialized.

Q45 | SYNTAX | *Pointer Initialization*

Which of the following is the correct syntax to declare a pointer to a double variable and initialize it to point to variable `d`?

A double dp = *d;

B dp double = &d;

C double *dp = d;

D double *dp = &d;

Ans: D

Explanation: This correctly combines declaration and initialization using the address-of operator.

Q46 | ASSERTION-REASON | *Syntax*

Assertion (A): The syntax `int* p;` and `int *p;` both declare p as a pointer to int.
Reason (R): In a single-variable declaration, whitespace around the * symbol does not affect its binding.

A A is false, R is true

B Both A and R are true, and R correctly explains A

C Both A and R are false

D A is true, R is false

Ans: B

Explanation: Spacing around * is cosmetic in a single declaration; both forms declare an equivalent pointer.

Chapter 4: Built-in Address Operator (&)

14 MCQs

Q47 | **DEFINITION** | ***Address-of Operator***

What does the & operator do when applied to a variable?

A Declares the variable as a pointer

B Returns the value stored in the variable

C Returns the memory address of the variable

D Performs a bitwise AND with itself

Ans: C

Explanation: & is the address-of operator, returning the memory location of its operand.

Q48 | **CONCEPT** | ***Purpose***

The & operator is classified as a:

A Ternary operator

B Logical operator

C Unary operator

D Binary operator

Ans: C

Explanation: The address-of operator (&) takes exactly one operand.

Q49 | **CODE SNIPPET** | ***Usage***

In `scanf("%d", &num);`, the & operator is used to:

A Multiply num by an unknown factor

B Pass the address of num so scanf can store the input value there

C Compare num with another variable

D Permanently declare num as a pointer

Ans: B

Explanation: scanf needs the variable's address to write the entered value directly into it.

Q50 | CODE SNIPPET | *Usage*

Which of the following correctly prints the address of an int variable `x`?

A printf("%d", x);

B printf("%p", *x);

C printf("%p", &x);

D printf("%p", x);

Ans: C

Explanation: &x yields the address, and %p is the appropriate format specifier for it.

Q51 | APPLICATION | *Applications*

The result of applying & to a variable is best stored in:

A A normal variable of the same base type

B An array of characters

C A pointer variable of a compatible type

D A floating-point variable, regardless of the original type

Ans: C

Explanation: An address value should be captured in a properly typed pointer variable.

Q52 | STATEMENT | *Syntax*

Which statement about the & operator is TRUE?

A It always returns a value of type int

B It cannot be applied to a constant literal (e.g., &5 is invalid)

C It can be applied to any literal value, such as &5

D It changes the value of the variable it is applied to

Ans: B

Explanation: & requires an addressable lvalue; constant literals do not have a storage location, so &5 is invalid.

Q53 | CODE SNIPPET | *Memory Address Retrieval*

In the expression `p = &a;`, if `a` is declared as `float`, then `p` must be declared as:

A char *p;

B float p;

C float *p;

D int *p;

Ans: C

Explanation: The pointer's declared type must match the type of variable whose address it holds.

Q54 | TRICKY | *Purpose*

Applying & directly to the result of another & (like &&a as address-of-address, without an intermediate pointer) is:

A The standard way to declare double pointers

B Equivalent to using the logical AND operator

C Invalid, since the result of & is not itself an addressable lvalue

D Perfectly valid and gives the address of the address

Ans: C

Explanation: & requires an lvalue operand; the result of & is a value, not storage, so applying & to it again is invalid.

Q55 | CONCEPT | *Applications*

The address-of operator is primarily used together with which other concept to enable indirect access?

A Pointer variables

B Loops

C Arrays only

D Functions returning void

Ans: A

Explanation: & supplies the address that a pointer variable then stores for indirect access.

Q56 | CODE SNIPPET | *Applications*

Which of the following retrieves the address of the first element of array `arr[5]`?

A &arr[0]

B *arr[0]

C arr[0]

D &&arr

Ans: A

Explanation: &arr[0] explicitly gives the address of the element at index 0.

Q57 | CODE SNIPPET | *Address-of Operator*

Given `int a = 5, *p = &a;`, what does `&a` represent?

A The size of variable a

B The value 5 itself

C The address of pointer p

D The memory address where a's value (5) is stored

Ans: D

Explanation: & applied to a yields its memory address, which is assigned to p.

Q58 | APPLICATION | *Applications*

Which of the following is a classic application of the address-of operator in C programs?

A Converting an integer to a floating-point number

B Performing arithmetic operations on the variable's value

C Declaring a new function

D Passing a variable's address to a function so it can modify the original value

Ans: D

Explanation: Passing &variable allows a function to alter the caller's original data through the received address.

Q59 | ASSERTION-REASON | *Applications*

Assertion (A): The & operator can be applied to an already declared pointer variable.
Reason (R): & retrieves the address at which the pointer variable itself is stored in memory.

A Both A and R are true, and R correctly explains A

B A is false, R is true

C A is true, R is false

D Both A and R are false

Ans: A

Explanation: &p gives the address of pointer p itself, which is a valid and meaningful operation.

Q60 | DEFINITION | *Address-of Operator*

Which symbol is used exclusively as the address-of operator in standard C?

A #

B &

C *

D @

Ans: B

Explanation: & is C's designated address-of operator.

Chapter 5: Pointers to Variables of Different Data Types

16 MCQs

Q61 | CONCEPT | *Integer Pointers*

A pointer declared as `int *ip;` is best suited to point to a variable of type:

A float
B double
C int
D char

Ans: C

Explanation: An int pointer is intended to point to and correctly interpret int-typed data.

Q62 | SYNTAX | *Character Pointers*

Which pointer declaration is appropriate for pointing to a character variable?

A char *cp;
B double *cp;
C int *cp;
D float *cp;

Ans: A

Explanation: A character pointer must be declared with the char base type.

Q63 | CONCEPT | *Float Pointers*

A float pointer (`float *fp;`) is used to:

A Store the address of an int variable only
B Store a float value directly, without needing an address
C Automatically convert integers into floats
D Store the address of a float variable and interpret data at that address as float

Ans: D

Explanation: The float pointer's base type governs how the data at its address is read.

Q64 | CONCEPT | *Double Pointers*

In this syllabus, a pointer declared as `double *dp;` refers to:

A A variable that stores two double values

B A pointer to a pointer variable in every case

C A pointer that always points to two variables

D A pointer meant to point to a double-type variable

Ans: D

Explanation: Here, 'double pointer' means a pointer to a double, distinct from the separate pointer-to-pointer concept.

Q65 | TRICKY | *Pointer Compatibility*

Assigning the address of a `char` variable directly to an `int *` pointer (without a cast) is generally:

A Automatically converted without issue in standard C

B Always completely safe and recommended

C Considered a type mismatch that may trigger a compiler warning/error

D Required for character pointers to function correctly

Ans: C

Explanation: Pointer assignment normally requires matching (or compatible) base types.

Q66 | CODE SNIPPET | *Character Pointers*

Which of these correctly declares and initializes a character pointer to variable `ch`?

A char ch = 'A', cp = &ch;

B char ch = 'A', *cp = &ch;

C char *cp = ch;

D char cp = &ch;

Ans: B

Explanation: cp must be declared with * and assigned ch's address using &.

Q67 | CONCEPT | *Applications*

Pointer 'compatibility' in C generally means:

A A pointer can point to any data type interchangeably without issue

B Compatibility refers to matching variable names

C A pointer variable should point to a variable of the same (or compatible) base data type

D Compatibility refers only to arithmetic operators

Ans: C

Explanation: Type compatibility ensures the pointer correctly interprets the data it points to.

Q68 | CONCEPT | *Float Pointers*

When a float pointer is dereferenced, the compiler treats the data at that address as:

A A float value

B A char value, always

C A string, always

D An int value, always

Ans: A

Explanation: Dereferencing interprets the bytes at the address according to the pointer's declared base type.

Q69 | CODE SNIPPET | *Double Pointers*

Which of the following is the correct way to reference a double variable `d` using a pointer `dp`?

A double d, *dp = &d;

B double d, dp = &d;

C double *dp = d;

D double dp = d;

Ans: A

Explanation: dp must be declared as a pointer and assigned &d.

Q70 | TRICKY | *Applications*

Using an `int *` pointer to point to a `double` variable and then dereferencing it will typically:

A Automatically convert the double value to int correctly

B Cause the double's value to be doubled

C Work perfectly fine on all systems

D Produce incorrect/unexpected results because of differing sizes and interpretation

Ans: D

Explanation: Mismatched pointer types misinterpret the underlying bytes, leading to incorrect results.

Q71 | CONCEPT | *Character Pointers*

Which pointer type would you use to point to a variable declared as `char grade;`?

A int *

B float *

C char *

D double *

Ans: C

Explanation: A char variable requires a char pointer for correct interpretation.

Q72 | CONCEPT | *Applications*

The concept of "pointer compatibility" mainly ensures that:

A Pointers always occupy sizes based only on the variable's current value

B Only one pointer type can exist in a single program

C Pointers of any type can be freely interchanged with no side effects

D Dereferencing a pointer accesses the correct number of bytes with the correct interpretation

Ans: D

Explanation: Compatibility guarantees correct byte-count and interpretation on dereferencing.

Q73 | STATEMENT | *Generic Usage*

Which of the following statements is TRUE regarding pointers to different data types?

A A char pointer cannot point to a char variable

B Each pointer type is intended to correctly interpret the data at the address it points to, based on its declared type

C Data type has no effect on pointer usage

D All pointer types behave identically regardless of declared type

Ans: B

Explanation: The declared base type of a pointer determines how the pointed-to data is interpreted.

Q74 | CODE SNIPPET | *Variable Referencing*

Given `float f = 3.14, *fp = &f;`, what is the variable being referenced (pointed to) in this statement?

A fp

B f

C &f

D 3.14

Ans: B

Explanation: fp is assigned the address of f, so f is the referenced variable.

Q75 | APPLICATION | *Applications*

Which of these applications commonly uses character pointers?

A Declaring loop counters

B Performing floating-point arithmetic directly

C Storing large numeric calculations

D Handling strings (arrays of characters)

Ans: D

Explanation: Character pointers are the standard mechanism used to represent and manipulate strings in C.

Q76 | ASSERTION-REASON | *Pointer Compatibility*

Assertion (A): A pointer of type `int *` can safely and portably store the address of a `double` variable without a cast.
Reason (R): All pointer types in C are treated identically by the compiler.

A Both A and R are false

B Both A and R are true, and R explains A

C A is true, R is false

D A is false, R is true

Ans: A

Explanation: Pointer types are NOT treated identically; assigning across incompatible types without a cast is neither safe nor portable, so both statements are false.

Chapter 6: Indirection Operator (*)

16 MCQs

Q77 | **DEFINITION** | ***Dereference Operator***

The * operator, when used with an already-declared pointer variable in an expression (not in a declaration), is called the:

A Address-of operator

B Multiplication operator

C Indirection/dereference operator

D Assignment operator

Ans: C

Explanation: In expressions, * applied to a pointer dereferences it to access the pointed-to value.

Q78 | **CODE SNIPPET** | ***Reading Values***

What does `*p` give, if `p` is a pointer holding the address of variable `a`?

A The address of variable a

B The value stored in variable a

C The address of pointer p

D The size of variable a

Ans: B

Explanation: Dereferencing p retrieves the value at the address it holds, which is a's value.

Q79 | **CODE SNIPPET** | ***Reading Values***

Given `int a = 10, *p = &a;`, what is the value of `*p`?

A The address of p

B 10

C The address of a

D Undefined

Ans: B

Explanation: p points to a, so *p evaluates to a's current value, 10.

Q80 | CODE SNIPPET | *Modifying Values*

Which statement modifies the value of `a` using pointer `p` (where `p = &a`)?

A a = *p + p;

B &p = 20;

C p = 20;

D *p = 20;

Ans: D

Explanation: *p = 20; dereferences p and assigns 20 to the memory location it points to, i.e., a.

Q81 | CODE SNIPPET | *Modifying Values*

In `int a=5, *p=&a; *p = *p + 1;`, after execution, the value of `a` becomes:

A The address of a, incremented

B 6

C Undefined

D 5

Ans: B

Explanation: *p reads a's current value (5), adds 1, and stores it back at a's address, making a equal to 6.

Q82 | CONCEPT | *Purpose*

The indirection operator is classified as a:

A Ternary operator

B Binary operator

C Unary operator

D Relational operator

Ans: C

Explanation: The dereference operator (*) takes a single operand — the pointer.

Q83 | SYNTAX | *Syntax*

Which of the following is the correct symbol used for dereferencing a pointer in C?

A &

B #

C *

D %

Ans: C

Explanation: * is the indirection/dereference operator.

Q84 | CODE SNIPPET | *Reading Values*

Given `int x = 15, *p = &x; printf("%d", *p);`, the output will be:

A A garbage value

B The address of x

C 15

D A compilation error

Ans: C

Explanation: *p dereferences p to print x's value, which is 15.

Q85 | DEFINITION | *Purpose*

Using * to read the value pointed to by a pointer is called:

A Addressing

B Declaring

C Dereferencing

D Casting

Ans: C

Explanation: This action of accessing the pointed-to value is termed dereferencing.

Q86 | TRICKY | *Applications*

Applying * to a pointer that has NOT been properly initialized (a wild pointer) is:

A Undefined behavior, which may crash the program or return a garbage value

B A syntax error caught at compile time

C Always guaranteed to return the pointer's own address

D Always guaranteed to return 0

Ans: A

Explanation: Dereferencing an uninitialized pointer accesses an arbitrary memory location, leading to undefined behavior.

Q87 | CODE SNIPPET | *Modifying Values*

Given `float f = 2.5, *fp = &f; *fp = *fp * 2;`, after execution the value of `f` is:

A 5.0

B 2.5

C 0.0

D A compilation error

Ans: A

Explanation: *fp reads f (2.5), doubles it, and stores 5.0 back at f's address.

Q88 | CODE SNIPPET | *Reading Values*

Which of the following correctly reads and prints the value pointed to by an int pointer `p`?

A printf("%d", &p);

B printf("%d", p);

C printf("%d", *p);

D printf("%p", *p);

Ans: C

Explanation: *p gives the pointed-to int value, correctly matched with the %d specifier.

Q89 | STATEMENT-BASED | *Applications*

'The indirection operator can also be used to modify the value at the address a pointer points to.' This statement is:

A False, only the & operator can modify values

B False, pointers are read-only

C True, for all compatible pointer types

D True, but only for int pointers

Ans: C

Explanation: *p = value works for any pointer type pointing to a modifiable variable of a compatible type.

Q90 | CODE SNIPPET | *Modifying Values*

Given `int a=1, b=2, *p=&a, *q=&b; *p = *q;`, after execution the value of `a` becomes:

A Undefined

B 1

C The address of b

D 2

Ans: D

Explanation: *q reads b's value (2), which is then stored at the location p points to (a), making a equal to 2.

Q91 | COMPARISON | *Purpose*

Which best describes the relationship between the & and * operators?

A Both always return a result of type int

B They are complementary/inverse operators — & obtains an address, * obtains the value at an address

C & is used only with arrays, * only with pointers to functions

D They perform the same operation

Ans: B

Explanation: & and * work as inverses of one another with respect to addresses and values.

Q92 | TRICKY | *Reading Values*

In the expression `*(&a)`, the result is equivalent to:

A Always 0

B a (the value of variable a itself)

C The address of a pointer to a

D The address of a

Ans: B

Explanation: Dereferencing the address-of a variable returns the variable's own value, undoing the & operation.

Chapter 7: Array Name as a Pointer Variable

14 MCQs

Q93 | CONCEPT | *Array Base Address*

In C, the name of an array (without any subscript) represents:

A The total number of elements in the array

B The value of the last element

C The base address (address of the first element) of the array

D The sum of all array elements

Ans: C

Explanation: An array name, when used in most expressions, decays to the address of its first element.

Q94 | CODE SNIPPET | *Array Base Address*

Given `int arr[5];`, the expression `arr` is equivalent to:

A arr[0]

B arr[5]

C &arr[0]

D &arr[5]

Ans: C

Explanation: The array name decays to the address of index 0, i.e., &arr[0].

Q95 | COMPARISON | *Pointer Representation*

Unlike a normal pointer variable, the array name:

A Stores the value of the last element

B Can be freely reassigned like any pointer

C Automatically increases in size as elements are added

D Is a constant and cannot be reassigned to point elsewhere

Ans: D

Explanation: An array name is a non-modifiable lvalue, so it cannot be made to point to a different location.

Q96 | CODE SNIPPET | *Access Concepts*

Given `int arr[5] = {10,20,30,40,50};`, what does `*arr` give?

A 50

B 10

C Undefined

D The base address of arr

Ans: B

Explanation: *arr dereferences the array name (its base address), giving the first element's value, 10.

Q97 | CODE SNIPPET | *Access Concepts*

Given `int arr[5];`, the expression `*(arr+2)` accesses:

A arr[2]

B arr[3]

C arr[0]

D The address of arr[2]

Ans: A

Explanation: Adding 2 to the base address and dereferencing gives the element at index 2.

Q98 | STATEMENT | *Relationship between Array Name and Pointer*

Which statement is TRUE about array names and pointers?

A Array names cannot be used in pointer expressions

B An array name is identical to a pointer variable in every respect, including reassignability

C Array names always store a copy of the entire array's data

D An array name behaves like a pointer to its first element but is not itself a reassignable pointer variable

Ans: D

Explanation: Array names decay to pointers for many purposes, but remain constant addresses, unlike true pointer variables.

Q99 | CODE SNIPPET | *Access Concepts*

Given `int arr[4]; int *p = arr;`, this statement is:

A Invalid — requires the & operator before arr

B Invalid — arrays cannot be assigned to pointers

C Valid — p now points to the first element of arr

D Valid, but p points to the last element

Ans: C

Explanation: Since arr decays to its base address, it can be directly assigned to a compatible pointer.

Q100 | TRICKY | *Characteristics*

Since `arr` decays to a pointer in most expressions, `sizeof(arr)` used in the same scope as its declaration gives:

A Only the number of elements, with no size information

B The total size of the array in bytes (not the size of a single pointer)

C Always the size of a single pointer (e.g., 8 bytes)

D Always exactly 4 bytes regardless of array type

Ans: B

Explanation: sizeof applied directly to an array (in its declaring scope) evaluates the array's full size, since it does not decay in this context.

Q101 | CONCEPT | *Access Concepts*

The expression `arr[i]` is internally treated by the compiler as equivalent to:

A (arr * i)

B &(arr + i)

C *(arr - i)

D *(arr + i)

Ans: D

Explanation: Array subscripting is defined in terms of pointer arithmetic and dereferencing.

Q102 | **STATEMENT** | ***Array Base Address***

Which of the following about the array's base address is TRUE?

A The base address is the address of the array's first (index 0) element

B The base address is the address of the last element

C The base address is always numerically 0

D The base address changes each time the array is accessed

Ans: A

Explanation: The base address is a fixed reference point: the location of index 0.

Q103 | **CODE SNIPPET** | ***Access Concepts***

Given `char str[] = "HI";`, what does the expression `str` represent?

A The address of the first character 'H'

B The total length of the string

C The character 'H' itself

D The last character '\0'

Ans: A

Explanation: As an array name, str decays to the address of its first character.

Q104 | **COMPARISON** | ***Similarities***

Which similarity exists between array names and pointer variables?

A Both occupy exactly one byte of memory

B Both can be reassigned to point to different memory locations

C Both are automatically initialized to NULL

D Both can be used with the * operator to access the value at an address

Ans: D

Explanation: Dereferencing works on both array names (via decay) and pointer variables.

Q105 | **CODE SNIPPET** | ***Access Concepts***

Given `int a[3] = {1,2,3}, *p = a;`, evaluating `p == a` is:

A True only after incrementing p once

B True, since both hold the same base address

C False, they can never be equal

D A compile-time error

Ans: B

Explanation: p is initialized to a's base address, so the comparison is true.

Q106 | ASSERTION-REASON | *Array Base Address*

Assertion (A): `arr + 1` (where arr is an int array) advances the address by 1 byte.
Reason (R): Pointer arithmetic moves by the size of one element of the pointer's data type, not by a fixed 1 byte.

A Both A and R are true, and R explains A

B A is false, R is true

C Both A and R are false

D A is true, R is false

Ans: B

Explanation: arr+1 actually moves by sizeof(int) bytes, making the Assertion false, while the Reason correctly states the general rule.

Chapter 8: Pointers and Arrays

16 MCQs

Q107 | **STATEMENT** | ***Relationship***

Which of the following statements about pointers and arrays is TRUE?

A Pointers cannot be used to traverse arrays

B Array elements can only be accessed using indexing, never pointers

C Array elements can be accessed using both array indexing and pointer notation

D Arrays and pointers are always identical data types

Ans: C

Explanation: C allows equivalent access to array elements via a[i] or *(p+i).

Q108 | **CODE SNIPPET** | ***Element Referencing***

Given `int a[5]={1,2,3,4,5}, *p=a;`, which expression accesses the 3rd element (index 2, value 3)?

A p+2

B *p+2

C *(p+2)

D *(p+3)

Ans: C

Explanation: Adding 2 to p and dereferencing accesses index 2, giving the value 3.

Q109 | **COMPARISON** | ***Differences***

A key DIFFERENCE between an array name and a pointer variable is:

A Pointers cannot be incremented, but array names can

B Both are identical with no difference at all

C An array name can store any address, but pointer variables are fixed

D An array name is a constant address, while a pointer variable can be reassigned to point elsewhere

Ans: D

Explanation: This constancy vs reassignability is the core distinguishing feature.

Q110 | CODE SNIPPET | *Pointer Access*

Given `int a[4] = {2,4,6,8};` and `int *p = a;`, evaluating `p++;` then `*p` gives:

A 6

B The address of a[1]

C 4

D 2

Ans: C

Explanation: p++ advances p to a[1] (value 4), so *p now yields 4.

Q111 | CONCEPT | *Array Traversal*

To traverse all elements of array `a[5]` using a pointer `p` initialized as `p = a`, the typical loop condition uses:

A p == 5

B p > a - 5

C p != NULL only

D p < a + 5 (comparing against one-past-the-end address)

Ans: D

Explanation: Comparing the pointer against the base address plus the element count is the standard traversal bound.

Q112 | TRICKY | *Relationship*

Which of the following is FALSE regarding arrays and pointers?

A Both indexing and pointer dereferencing can access the same element

B Array elements can be accessed via pointer arithmetic

C An array pointer, once decayed, can cause the original array's declared size to grow dynamically

D The array name gives the base address

Ans: C

Explanation: Pointer decay does not change the array's fixed, compile-time-declared size; this statement is false.

Q113 | CODE SNIPPET | *Element Referencing*

Given `int a[3]={5,10,15}, *p=a;`, what does `p[1]` return?

A 10

B The address of a[1]

C 5

D 15

Ans: A

Explanation: Pointer indexing p[1] is equivalent to *(p+1), giving the second element, 10.

Q114 | COMPARISON | *Similarities*

The similarity between `a[i]` and `*(p+i)` (where p points to array a) is that:

A Both expressions access the same array element at index i

B a[i] works only for char arrays

C They access different elements

D *(p+i) always causes an error

Ans: A

Explanation: Both notations are equivalent ways of referring to the same element.

Q115 | CONCEPT | *Pointer Access*

When traversing an array using a pointer, incrementing the pointer (`p++`) moves it to:

A The same element again

B The previous element in the array

C A completely random memory address

D The next element in the array (advancing by the size of one element)

Ans: D

Explanation: p++ advances the pointer by one element's worth of bytes.

Q116 | TRICKY | *Differences*

Given `int a[5], *p = a;`, which statement is INVALID?

A p++;

B p = a;

C a = p; (attempting to reassign the array name)

D p = a + 2;

Ans: C

Explanation: Array names are non-modifiable lvalues and cannot be reassigned, unlike pointer variables.

Q117 | CODE SNIPPET | *Element Referencing*

Element referencing using pointer notation `*(a+i)` is equivalent in value to:

A &a[i]

B a[i-1]

C a[i]

D a[i+1]

Ans: C

Explanation: This is the defining equivalence between pointer notation and array indexing.

Q118 | CONCEPT | *Relationship*

A key relationship between arrays and pointers in C is that:

A Arrays are stored internally as linked lists of pointers

B Arrays cannot interact with pointers at all

C The array name decays into a pointer to its first element in most expressions

D Pointers are automatically converted into arrays

Ans: C

Explanation: This decay is the foundation of the close relationship between arrays and pointers in C.

Q119 | CODE SNIPPET | *Element Referencing*

Given `int a[4]={1,2,3,4}, *p=&a[2];`, what does `*(p-1)` give?

A 1

B 4

C 2

D 3

Ans: C

Explanation: p points to a[2]; moving back one element points to a[1], value 2.

Q120 | CODE SNIPPET | *Array Traversal*

Which loop correctly prints all elements of `int a[5]` using pointer `p`, starting with `p = a`?

A for(int i=0;i<5;i++) printf("%d", *(p+i));

B for(int i=0;i<5;i++) printf("%d", a[p]);

C for(int i=5;i>0;i--) printf("%d", *p+i);

D for(int i=0;i<5;i++) printf("%d", p+i);

Ans: A

Explanation: Dereferencing p+i for each i correctly prints every array element.

Q121 | ASSERTION-REASON | *Similarities*

Assertion (A): Pointer notation and array indexing are functionally interchangeable for accessing array elements.
Reason (R): The compiler translates a[i] internally into *(a+i).

A A is true, R is false

B Both A and R are true, and R correctly explains A

C Both A and R are false

D A is false, R is true

Ans: B

Explanation: Since a[i] is defined as *(a+i), the two notations are indeed interchangeable, and R explains why.

Q122 | CODE SNIPPET | *Element Referencing*

Given `char s[]="CAT"; char *p = s;`, what does `*(p+1)` output?

A '\0'

B 'A'

C 'C'

D 'T'

Ans: B

Explanation: p+1 points to the second character of "CAT", which is 'A'.

Chapter 9: Pointer Arithmetic

18 MCQs

Q123 | **CONCEPT** | ***Pointer Movement***

If `int *p` points to an array element, `p+1` advances the address by:

A sizeof(int) bytes

B 1 byte always

C 2 bytes always

D sizeof(float) bytes

Ans: A

> **Explanation:** Pointer arithmetic scales by the size of the pointed-to type.

Q124 | **TRICKY** | ***Invalid Operations***

Which arithmetic operation is INVALID between two pointers of the same type pointing into the same array?

A Subtracting the two pointers

B Multiplying the two pointers

C Comparing the two pointers

D Adding an integer to one pointer

Ans: B

> **Explanation:** Pointer multiplication is meaningless and not permitted; subtraction and comparison are valid for same-array pointers.

Q125 | **CODE SNIPPET** | ***Subtraction***

Given `int a[5], *p=&a[1], *q=&a[4];`, what does `q - p` give?

A 3 bytes, regardless of element type

B 12, as a raw byte difference

C Undefined in all cases

D 3 (the number of elements between them)

Ans: D

> **Explanation:** Pointer subtraction within the same array yields the element count between the two addresses, not raw bytes.

Q126 | CONCEPT | *Valid Operations*

Which of the following pointer arithmetic operations is generally considered VALID and meaningful?

A Multiplying a pointer by a float

B Dividing two pointers

C Subtracting a smaller pointer from a larger pointer within the same array

D Adding two pointers together

Ans: C

Explanation: Only subtraction (within the same array), addition of an integer, and comparison are meaningful pointer operations.

Q127 | CODE SNIPPET | *Addition*

Given `int a[5]={1,2,3,4,5}, *p=a; p = p + 3;`, what does `*p` give?

A 1

B 4

C 3

D 5

Ans: B

Explanation: p+3 points to index 3 (value 4).

Q128 | CONCEPT | *Pointer Movement*

For a `char *p`, incrementing `p` by 1 moves the address forward by:

A 4 bytes always

B 1 byte (the size of char)

C 2 bytes always

D 8 bytes always

Ans: B

Explanation: char has a size of 1 byte, so pointer arithmetic steps by 1 byte.

Q129 | CONCEPT | *Pointer Movement*

For a `double *dp`, `dp + 1` typically advances the address by:

A sizeof(int) bytes

B sizeof(double) bytes (commonly 8 bytes)

C 4 bytes always regardless of type

D 1 byte

Ans: B

Explanation: Pointer arithmetic on a double pointer scales by the size of a double.

Q130 | CONCEPT | *Pointer Arithmetic Rules*

Pointer arithmetic rule: Adding an integer `n` to a pointer `p` results in:

A p being advanced by exactly n bytes, regardless of type

B p being advanced by n elements (n times the size of the pointed type)

C p's stored value being multiplied by n

D A syntax error in all cases

Ans: B

Explanation: This scaling by element size is the defining rule of pointer arithmetic.

Q131 | TRICKY | *Invalid Operations*

Which of these is an INVALID pointer arithmetic operation in standard C?

A p1 - p2 (pointers into the same array)

B Adding two pointer variables together (p1 + p2)

C p++

D p + 5

Ans: B

Explanation: Adding two pointers has no defined meaning and is disallowed.

Q132 | CODE SNIPPET | *Decrement*

Given `int a[6], *p=&a[5];`, evaluating `p--;` moves the pointer to point at:

A a[4]

B a[0]

C a[6]

D a[5]

Ans: A

Explanation: Decrementing the pointer moves it back by one element.

Q133 | CONCEPT | *Valid Operations*

Pointer arithmetic is meaningful primarily when pointers point to:

A Constant literals only

B Elements within the same array (a contiguous memory block)

C Completely unrelated memory blocks

D Different data types simultaneously

Ans: B

Explanation: Standard pointer arithmetic guarantees meaningful results only within the same array.

Q134 | CODE SNIPPET | *Addition*

Given `float f[3], *fp=&f[0]; fp += 2;`, `fp` now points to:

A f[0]

B f[1]

C f[2]

D f[3]

Ans: C

Explanation: fp += 2 advances fp by two float elements, landing on f[2].

Q135 | CONCEPT | *Pointer Arithmetic Rules*

Which statement correctly explains why pointer arithmetic differs from normal integer arithmetic?

A Because pointers cannot be added or subtracted at all

B Because pointer arithmetic always results in a float value

C Because pointers ignore the data type entirely during arithmetic

D Because pointer arithmetic scales the operation by the size of the data type the pointer points to

Ans: D

Explanation: The key distinction is the type-based scaling applied to pointer operations.

Q136 | TRICKY | *Invalid Operations*

Which of the following is generally NOT allowed in pointer arithmetic?

A Comparing two pointers using relational operators

B Incrementing a pointer using ++

C Subtracting an integer from a pointer

D Multiplying a pointer with an integer

Ans: D

Explanation: Multiplication of a pointer is not a defined or valid pointer arithmetic operation.

Q137 | CODE SNIPPET | *Subtraction*

Given `int a[4]={10,20,30,40}, *p=a+3;`, what does `*(p-2)` return?

A 20

B 30

C 10

D 40

Ans: A

Explanation: p points to a[3]; moving back 2 elements reaches a[1], value 20.

Q138 | CONCEPT | *Subtraction*

The result of subtracting two pointers pointing into the same array is of type:

A A character

B A float value

C A pointer type

D An integer representing the number of elements between them

Ans: D

Explanation: Pointer subtraction yields an integer (element count), not another pointer.

Q139 | CODE SNIPPET | *Addition*

Given `char *s = "HELLO"; s += 2;`, what does `*s` output?

A 'H'

B 'E'

C 'L'

D 'O'

Ans: C

Explanation: s+2 points to the third character of "HELLO", which is 'L'.

Q140 | ASSERTION-REASON | *Valid Operations*

Assertion (A): Pointer arithmetic on a `void *` pointer is straightforward and follows the same rules as `int *`.
Reason (R): Since `void` has no defined size, standard C does not permit direct arithmetic on `void *` pointers without a cast.

A A is true, R is false

B A is false, R is true

C Both A and R are true, and R explains A

D Both A and R are false

Ans: B

Explanation: Arithmetic on void* is not standard because void has no defined size, making the Assertion false while the Reason is correct.

Chapter 10: Unary Operators (++, --) with Pointers

14 MCQs

Q141 | CODE SNIPPET | *Usage with Pointers*

Given `int a[5], *p=a; p++;`, the pointer `p` now points to:

A a[0]

B a[1]

C a[2]

D a[-1]

Ans: B

Explanation: p++ advances the pointer by one element, from a[0] to a[1].

Q142 | CONCEPT | *Postfix Increment*

In the expression `*p++`, the order of operations is:

A p is incremented first, then dereferenced

B Both operations happen simultaneously with no defined order

C The expression is a syntax error

D The current value pointed to by p is accessed first, then p is incremented (postfix)

Ans: D

Explanation: Postfix ++ has higher precedence than * for binding, but its side effect (incrementing p) applies after the value is used.

Q143 | CONCEPT | *Prefix Increment*

In the expression `*++p`, the order of operations is:

A p is incremented first, then the new location is dereferenced (prefix)

B The current value is accessed first, then p is incremented

C This expression is invalid in C

D Both operators cancel each other out

Ans: A

Explanation: Prefix ++ increments p before the dereference occurs.

Q144 | CODE SNIPPET | *Postfix Increment*

Given `int a[3]={10,20,30}, *p=a;`, what does `*p++` evaluate to (the value returned by the expression)?

A 10

B 20

C The address of a[1]

D 30

Ans: A

Explanation: The current value at p (10) is used first; p is then incremented afterward.

Q145 | CODE SNIPPET | *Prefix Increment*

Given `int a[3]={10,20,30}, *p=a;`, what does `*++p` evaluate to?

A 10

B 20

C 30

D The address of a[0]

Ans: B

Explanation: p is incremented to a[1] first, then dereferenced to give 20.

Q146 | CONCEPT | *Postfix Decrement*

Postfix decrement on a pointer, `p--`, does which of the following?

A Deletes the value pointed to

B Uses the pointer's current value in the expression, then moves p to the previous element

C Moves p first, then uses the new value

D Has no effect on the pointer's address

Ans: B

Explanation: Postfix decrement applies its effect after the current value is used.

Q147 | CONCEPT | *Prefix Decrement*

Prefix decrement on a pointer, `--p`, does which of the following?

A Increments the pointer instead

B Only works with float pointers

C Moves p to the previous element first, then that new value is used in the expression

D Uses the current value first, then decrements

Ans: C

Explanation: Prefix decrement applies its effect before the value is used.

Q148 | CODE SNIPPET | *Usage with Pointers*

Given `int a[4]={1,2,3,4}, *p=&a[3];`, what does `*p--` return (the value returned by the expression)?

A 4

B 3

C 1

D 2

Ans: A

Explanation: The current value (a[3]=4) is used first, then p is decremented afterward.

Q149 | CODE SNIPPET | *Usage with Pointers*

After `*p--;` executes in the previous scenario (starting at a[3]), where does p now point?

A a[1]

B a[2]

C a[3]

D a[0]

Ans: B

Explanation: The postfix decrement moves p back by one element after use, from a[3] to a[2].

Q150 | TRICKY | *Applications*

Which statement is TRUE about using ++ or -- directly on array names (not pointer variables)?

A It is invalid, since array names are constant and cannot be incremented or decremented

B It is valid and moves the array's base address

C It deletes the first array element

D It resizes the array

Ans: A

Explanation: Array names are non-modifiable lvalues, so applying ++ or -- to them is not allowed.

Q151 | CODE SNIPPET | *Usage with Pointers*

Given `char *s = "AB"; printf("%c", *s++); printf("%c", *s);`, the output is:

A BB

B BA

C AB

D AA

Ans: C

Explanation: The first call prints 'A' and advances s; the second call then prints the new current character, 'B'.

Q152 | TRICKY | *Applications*

The expression `++*p` (as opposed to `*++p`) means:

A Move the pointer to the next address

B This is an invalid expression in C

C Increment the value stored at the address p points to (the pointer itself does not move)

D Increment the pointer's own address value

Ans: C

Explanation: ++*p applies increment to the dereferenced value, not to the pointer.

Q153 | CODE SNIPPET | *Usage with Pointers*

Given `int a=5, *p=&a; ++*p;`, what is the value of `a` after execution?

A 6

B A compilation error

C The address of a, incremented

D 5

Ans: A

Explanation: ++*p increments the value at the address p points to, so a becomes 6.

Q154 | ASSERTION-REASON | *Usage with Pointers*

Assertion (A): `*p++` and `(*p)++` behave identically.
Reason (R): Postfix ++ binds to p (not to the dereferenced value) in the expression `*p++`, since it has higher precedence than the unary * for binding purposes.

A Both A and R are true, and R explains A

B A is false, R is true

C A is true, R is false

D Both A and R are false

Ans: B

Explanation: *p++ increments the pointer p, while (*p)++ increments the pointed-to value — these are different operations, so A is false while R correctly explains the mechanism.

Chapter 11: One-Dimensional Arrays and Pointers

16 MCQs

Q155 | CONCEPT | *Pointer Traversal*

To traverse a 1D array using pointer notation, the pointer should typically be initialized to:

A The address of the last element only

B The base address of the array (e.g., p = arr;)

C NULL, in every case

D The value of the first element

Ans: B

Explanation: Traversal begins by pointing to the array's first element (its base address).

Q156 | CODE SNIPPET | *Pointer Notation*

Given `int a[5]={1,2,3,4,5};` and pointer `p=a`, which expression gives the sum of the first two elements?

A *(p+p+1)

B *p + *(p+1)

C *p + p+1

D p + p+1

Ans: B

Explanation: Dereferencing p and p+1 separately and adding gives the sum of a[0] and a[1].

Q157 | CONCEPT | *Pointer Notation*

Which pointer notation form is equivalent to array indexing `a[i]` for a 1D array?

A *(a+i)

B *a+i

C a*(i)

D *(a-i)

Ans: A

Explanation: This is the standard equivalence between indexing and pointer notation.

Q158 | TRICKY | *Indexing using Pointers*

Given a 1D array `int a[4]`, and pointer `p=a`, the expression `p+4` refers to:

A The address of a[0]

B The address one position past the last valid element (a valid end marker, though not dereferenceable)

C The address of a[3], the last valid element

D An invalid pointer expression in C

Ans: B

Explanation: C permits forming (but not dereferencing) a pointer one position past the end of an array, often used as a loop boundary.

Q159 | CONCEPT | *Pointer Traversal*

In pointer-based array traversal, incrementing the pointer variable in a loop is sometimes considered efficient because:

A It reduces the array's size in memory

B It always changes the array's actual stored values

C It works only for character arrays

D It can access the next element directly without recomputing an offset via multiplication

Ans: D

Explanation: Simple pointer increment avoids recalculating i * element_size on each iteration.

Q160 | CODE SNIPPET | *Array Access*

Given `int a[3]={7,8,9}, *p=a;`, which of these correctly prints all elements using pointer notation in a loop?

A for(int i=0;i<3;i++) printf("%d ", p+i);

B for(int i=0;i<3;i++) printf("%d ", *p);

C for(int i=0;i<3;i++) printf("%d ", &p[i]);

D for(int i=0;i<3;i++) printf("%d ", *(p+i));

Ans: D

Explanation: Dereferencing p+i for each i correctly prints each successive element's value.

Q161 | CONCEPT | *Relationship between Arrays and Pointers*

The relationship between a 1D array and a pointer variable pointing to it is that:

A The pointer automatically resizes along with the array

B The pointer can access every element of the array through address arithmetic based on the array's base address

C The pointer must always be declared as void *

D The pointer can only ever access the first element

Ans: B

Explanation: Given the base address, arithmetic on the pointer reaches every element.

Q162 | CODE SNIPPET | *Array Access*

Given `int a[5]={1,2,3,4,5}, *p=a; p=p+2;`, `*p` refers to:

A a[0], value 1

B a[4], value 5

C a[1], value 2

D a[2], value 3

Ans: D

Explanation: p+2 moves the pointer to index 2 of the array.

Q163 | CODE SNIPPET | *Pointer Notation*

Which of the following correctly uses pointer notation to access the last element of `int a[6]`?

A *(a+5)

B *(a-5)

C a[6]

D *(a+6)

Ans: A

Explanation: The last valid index is 5, accessed as *(a+5).

Q164 | CONCEPT | *Indexing using Pointers*

Pointer indexing `p[i]` (where p points to an array) is functionally equivalent to:

A p*i

B &p[i-1]

C *(p-i)

D *(p+i)

Ans: D

Explanation: p[i] is defined in C to mean *(p+i), regardless of whether p is an array name or pointer variable.

Q165 | COMPARISON | *Relationship between Arrays and Pointers*

Which statement correctly compares array indexing and pointer notation for 1D arrays?

A `*(a+i)` cannot be used for arrays, only for pointer variables

B Both `a[i]` and `*(a+i)` access the same memory location and produce identical results

C They access different memory locations

D `a[i]` is always faster than `*(a+i)` on every compiler

Ans: B

Explanation: The two notations are semantically equivalent for accessing array elements.

Q166 | CODE SNIPPET | *Array Access*

Given `int a[4]={2,4,6,8}, *p=a;`, evaluate: `*(p+1) + *(p+2)`

A 6

B 8

C 12

D 10

Ans: D

Explanation: *(p+1) is 4 and *(p+2) is 6; their sum is 10.

Q167 | FILL-IN-THE-BLANK | *Pointer Notation*

To point to the third element of array `a[10]` using pointer p (p = a), you write: `p = a ____ 2;`

A +

B -

C /

D *

Ans: A

Explanation: Adding 2 to the base address moves the pointer to index 2 (the third element).

Q168 | CONCEPT | *Pointer Traversal*

A pointer traversing a 1D array typically stops when:

A It reaches an address equal to (base address + number of elements), one past the last valid element

B It reaches the array's first element again

C It reaches address 0 in every case

D It is manually reset to NULL mid-loop

Ans: A

Explanation: The one-past-the-end address is the conventional loop termination boundary.

Q169 | CODE SNIPPET | *Pointer Traversal*

Given `int a[3]={100,200,300};`, which statement correctly initializes pointer p to point to the array?

A int *p = &a;

B int *p = a;

C int *p = a[];

D int p = *a;

Ans: B

Explanation: p = a assigns the decayed base address; &a has a different (array-pointer) type, not directly suited to simple element traversal.

Q170 | ASSERTION-REASON | *Relationship between Arrays and Pointers*

Assertion (A): `*(a+i)` and `a[i]` always yield the same result for a 1D array `a`.
Reason (R): The C compiler internally converts array subscript notation `a[i]` into pointer arithmetic `*(a+i)`.

A A is true, R is false

B Both A and R are false

C A is false, R is true

D Both A and R are true, and R correctly explains A

Ans: D

Explanation: Since a[i] is defined as *(a+i) by the language, both statements are true, with R explaining A.

Chapter 12: Array of Pointers

14 MCQs

Q171 | **DEFINITION** | ***Definition***

An array of pointers is defined as:

A A single pointer that stores multiple arrays

B An array that stores pointer sizes

C An array whose elements are themselves pointer variables

D An array that stores only the addresses of other arrays, never variables

Ans: C

Explanation: Each element of such an array is a pointer, capable of holding an address.

Q172 | **SYNTAX** | ***Declaration***

Which of the following correctly declares an array of 5 integer pointers?

A int *ptrArr(5);

B int *ptrArr[5];

C int ptrArr[5]*;

D pointer int ptrArr[5];

Ans: B

Explanation: This declares ptrArr as an array of 5 elements, each of type 'pointer to int'.

Q173 | **APPLICATION** | ***Applications***

A common application of an array of character pointers (`char *names[]`) is:

A Declaring loop counters only

B Performing floating-point calculations

C Storing and managing a list of strings efficiently

D Storing a single large number

Ans: C

Explanation: An array of char pointers is the classic way to represent a list of strings in C.

Q174 | CODE SNIPPET | *Basic Usage*

Given `int a=1,b=2,c=3; int *arr[3] = {&a,&b,&c};`, what does `*arr[1]` give?

A 3

B The address of b

C 1

D 2

Ans: D

Explanation: arr[1] holds &b; dereferencing it gives b's value, 2.

Q175 | CODE SNIPPET | *Basic Usage*

In `char *names[3] = {"Ravi","Meena","Om"};`, the expression `names[0]` gives:

A The character 'R'

B The entire array of all names combined

C The number 0

D The address of the first character of "Ravi"

Ans: D

Explanation: names[0] is a char pointer holding the address of the string "Ravi".

Q176 | STATEMENT | *Characteristics*

Which statement about arrays of pointers is TRUE?

A An array of pointers can only be declared for integers

B Each element of the array can point to a separate, independently located memory block

C An array of pointers cannot be initialized at the point of declaration

D All elements must point to consecutive memory addresses

Ans: B

Explanation: Unlike a regular array's contiguous data, each pointer element can independently reference any address.

Q177 | CODE SNIPPET | *Basic Usage*

To access the value pointed to by the second element of `int *arr[4]`, the correct expression is:

A arr*[1]

B *arr(1)

C *arr[1]

D arr[1]*

Ans: C

Explanation: arr[1] gives the pointer stored at index 1; * dereferences it to get the pointed-to value.

Q178 | APPLICATION | *Applications*

An array of pointers is particularly useful for handling:

A A constant that never changes

B A single character only

C A single fixed numeric value

D A collection of strings of varying lengths (an array of char pointers)

Ans: D

Explanation: This is the classic use case, since each string can have a different length without wasted space.

Q179 | CODE SNIPPET | *Basic Usage*

Given `int x=10, y=20, *p[2]; p[0]=&x; p[1]=&y;`, what does `*p[0] + *p[1]` evaluate to?

A 10

B 20

C Undefined

D 30

Ans: D

Explanation: *p[0] gives x (10) and *p[1] gives y (20); their sum is 30.

Q180 | CONCEPT | *Initialization*

Which of the following best describes the initialization of an array of pointers?

A All elements are automatically initialized to point to the same variable

B It must always be initialized only with numeric constants

C Each element must be assigned the address of a valid variable individually or via an initializer list

D Initialization is unnecessary; elements default to usable addresses

Ans: C

> **Explanation:** Each pointer element needs a valid address assigned, whether via an initializer list or individual assignment.

Q181 | CODE SNIPPET | *Basic Usage*

Given `char *arr[2] = {"Hi", "Bye"};`, `arr[1]` points to:

A The first character 'H' of "Hi"

B The first character 'B' of the string "Bye"

C The character count of the array

D NULL

Ans: B

> **Explanation:** arr[1] holds the address of the string literal "Bye", pointing at its first character.

Q182 | COMPARISON | *Characteristics*

What is the key characteristic that differentiates an "array of pointers" from a "pointer to an array"?

A A pointer to an array cannot be declared in C

B An array of pointers holds multiple separate addresses (one per element), whereas a pointer to an array holds one address referring to the whole array

C An array of pointers can only ever hold one address total

D They are exactly the same concept with different names

Ans: B

> **Explanation:** These are distinct concepts: multiple independent addresses versus a single address referring to an entire array.

Q183 | SYNTAX | *Declaration*

Which declaration correctly represents an array of 3 character pointers used to store 3 strings?

A char *str[3];

B char str[3];

C char str[3][];

D char *str;

Ans: A

Explanation: This declares str as an array of 3 elements, each a pointer to char, suitable for holding 3 string addresses.

Q184 | ASSERTION-REASON | *Characteristics*

Assertion (A): An array of pointers, `int *arr[5]`, occupies memory equal to 5 times the size of the int data it points to.
Reason (R): Each element of `arr` is itself a pointer, and a pointer's size is determined by the system's address width, not by the size of the pointed-to data type.

A A is false, R is true

B Both A and R are false

C A is true, R is false

D Both A and R are true, and R explains A

Ans: A

Explanation: The array itself occupies 5 times the pointer size (not the int size), making the Assertion false while the Reason correctly explains this.

Chapter 13: Command Line Arguments for main()

16 MCQs

Q185 | SYNTAX | *Syntax*

Which function signature correctly supports command-line arguments in C?

A int main(int argc, char *argv[])

B int main(char argc, int *argv[])

C void main(int argc)

D int main(int argv, char *argc[])

Ans: A

Explanation: This is the standard signature used to receive command-line arguments in main().

Q186 | DEFINITION | *argc*

In `int main(int argc, char *argv[])`, what does `argc` represent?

A The name of the program only

B The address of the first argument

C The count of command-line arguments, including the program name

D The total number of characters typed

Ans: C

Explanation: argc holds the total number of arguments passed, counting the program name as the first.

Q187 | CONCEPT | *argv*

In `int main(int argc, char *argv[])`, what does `argv[0]` typically contain?

A The first user-supplied argument

B Always NULL

C The total number of arguments

D The name (or path) of the executable program itself

Ans: D

Explanation: By convention, argv[0] is the invoked program's own name/path.

Q188 | CODE SNIPPET | *Command-line Argument Concepts*

If a program is run as `./prog hello world`, what is the value of `argc`?

A 3

B 0

C 1

D 2

Ans: A

Explanation: argc counts the program name plus the two arguments: 'prog', 'hello', 'world' = 3.

Q189 | CODE SNIPPET | *Command-line Argument Concepts*

For the command `./prog hello world`, what does `argv[1]` contain?

A NULL

B "./prog"

C "hello"

D "world"

Ans: C

Explanation: argv[1] is the first user-supplied argument after the program name, which is 'hello'.

Q190 | CONCEPT | *argv*

The data type of `argv` in `main(int argc, char *argv[])` is:

A An array of integers

B An array of floats

C A single character pointer

D An array of character pointers (equivalently, a pointer to a pointer to char)

Ans: D

Explanation: argv is an array of strings, i.e., char pointers, one per command-line token.

Q191 | TRICKY | *argv*

Which of the following is TRUE about `argv[argc]`?

A It is conventionally a NULL pointer, marking the end of the argument list

B Accessing it always causes a compile-time error

C It always holds the program's name

D It always equals argv[0]

Ans: A

Explanation: By convention, the array of argument pointers is NULL-terminated at index argc.

Q192 | CONCEPT | *Purpose*

Command-line arguments are primarily used to:

A Completely replace the need for input functions like scanf

B Automatically compile the program

C Declare global variables inside main()

D Pass input values/parameters to a program at the time of execution

Ans: D

Explanation: They let a user supply parameters to a program when launching it.

Q193 | STATEMENT | *argc*

Which statement about `argc` is TRUE?

A argc is always at least 1, since it includes the program name

B argc is always exactly equal to 10

C argc counts only numeric arguments

D argc can be 0 even when a program name is present

Ans: A

Explanation: Since the program name itself is always counted, argc is never less than 1.

Q194 | CODE SNIPPET | *Applications*

To print all command-line arguments, which loop is correct (given `argc`, `argv`)?

A for(int i=0;i<argc;i++) printf("%d ", argv[i]);

B for(int i=0;i<argc;i++) printf("%s ", argv[i]);

C for(int i=1;i<=argc;i++) printf("%s ", argv);

D for(int i=argc;i>0;i--) printf("%s ", argc[i]);

Ans: B

Explanation: Iterating i from 0 to argc-1 and printing argv[i] as a string correctly lists all arguments.

Q195 | TRICKY | *Syntax*

Which of the following about command-line argument syntax is CORRECT?

A Command-line arguments cannot be declared using pointer notation

B The second parameter can also be written as `char **argv` instead of `char *argv[]`

C The second parameter must always be written as `char argv[]` only

D argv must always be of type int

Ans: B

Explanation: char **argv and char *argv[] are equivalent parameter declarations for this purpose.

Q196 | CODE SNIPPET | *Command-line Argument Concepts*

If no extra arguments are passed when running a program (only the program name), the value of `argc` is:

A Undefined

B 1

C 2

D 0

Ans: B

Explanation: With only the program name present, argc equals 1.

Q197 | CONCEPT | *argv*

Each element `argv[i]` (for i from 0 to argc-1) is of type:

A int

B char (a single character)

C float *

D char * (a pointer to a null-terminated string)

Ans: D

Explanation: Each argument is represented as a string, accessed via a char pointer.

Q198 | APPLICATION | *Applications*

A practical application of command-line arguments is:

A Allowing a user to specify a filename to be processed when launching the program

B Automatically declaring all program variables

C Replacing the main() function entirely

D Preventing the program from compiling

Ans: A

Explanation: e.g., running ./prog data.txt lets the program receive 'data.txt' as an argument to process.

Q199 | SCENARIO | *Command-line Argument Concepts*

Given the command `./calc 10 20`, which statement correctly describes reading the arguments?

A argv[1] holds "10" and argv[2] holds "20" as strings, which may need conversion (e.g., using atoi) to use as numbers

B argv[0] holds "10"

C argv[1] and argv[2] are automatically integers, needing no conversion

D argc holds "10" and "20"

Ans: A

> **Explanation:** Command-line arguments always arrive as strings and typically need explicit conversion for numeric use.

Q200 | ASSERTION-REASON | *Purpose*

Assertion (A): Command-line arguments allow a C program to receive input without using scanf().
Reason (R): argv provides the arguments as an array of strings directly accessible in main(), captured at the time the program is launched.

A Both A and R are false

B A is false, R is true

C A is true, R is false

D Both A and R are true, and R correctly explains A

Ans: D

> **Explanation:** Since argv already contains the launch-time arguments as strings, no scanf call is needed to obtain them, and R correctly explains why.